

Revised Report — What Happens Through TrainingService

Overview

TrainingService is the offline industrial ML pipeline of the MDF1 ML platform.

Its responsibility is to transform raw MDF1 production/lab exports into:

- validated industrial datasets,
- engineered process features,
- trained ML models,
- explainability reports,
- and production-ready artifacts for PredictionService.

The pipeline was progressively refactored toward:

- centralized configuration,
- modular architecture,
- reduced redundancy,
- industrial traceability,
- production-aware evaluation,
- and explainable ML behavior.

High-Level Pipeline Flow

```
Raw MDF1 Excel Files
    ↓
Ingestion & Schema Parsing
    ↓
Validation
    ↓
Cleaning & Normalization
    ↓
Feature Engineering
    ↓
Audit / EDA
    ↓
Process Explainability Reports
    ↓
Time-Based Cross Validation
    ↓
Model Training & Hyperparameter Tuning
    ↓
Ensemble Evaluation
```

↓
Best Model Selection
↓
Artifact Publishing
↓
Final Explainability & Recommendation Reports

1. Raw MDF1 Data Ingestion

TrainingService loads all `.xlsx` files from:

`data/raw/`

The architecture evolved from:

- single hardcoded Excel file to:
- dynamic multi-file ingestion using wildcard patterns.

This allows:

- very large exports,
- split exports,
- future scalable ingestion.

The ingestion layer:

- reads KSoft exports,
 - parses multi-row headers,
 - extracts human-readable names,
 - extracts technical FQTS names,
 - builds the modeling dataframe,
 - generates column mappings,
 - saves flattened parquet/csv backups.
-

2. Validation Layer

Before modeling, the pipeline validates:

- required targets,
- feature existence,
- duplicate columns,
- invalid schemas,
- missing time columns,

- dataframe integrity.

This is important because industrial exports may contain:

- inconsistent schemas,
- operator-generated anomalies,
- mixed datatypes,
- missing process signals,
- incomplete lab data.

The validator acts as a protective industrial gate before ML training begins.

3. Cleaning & Data Normalization

The cleaning layer:

- normalizes text-based null values,
- parses timestamps,
- coerces numeric columns safely,
- removes invalid target rows,
- deduplicates production records,
- removes highly missing features.

The pipeline uses:

```
errors="coerce"
```

for industrial-safe conversion behavior.

This means:

- invalid values do not crash the pipeline,
- problematic values become `NaN`,
- processing continues safely.

This is especially important for MDF1/KSoft exports containing:

- `--`
- `NULL`
- broken timestamps
- mixed object values.

The parquet export layer was additionally hardened to safely serialize mixed industrial object columns.

4. Feature Engineering

The feature engineering layer transforms raw MDF process signals into higher-level process intelligence features.

Examples include:

Pressure Aggregates

```
pressPressureGlobal_mean  
pressPressureGlobal_std  
pressPressureFront_mean  
pressPressureMid_mean  
pressPressureEnd_mean  
pressPressureFrontEnd_diff
```

Temperature Aggregates

```
tempGlobal_mean  
tempGlobal_range
```

Thickness Aggregates

```
thicknessClosed_mean  
thicknessClosed_std  
thicknessClosed_range
```

These features better represent MDF process behavior than raw sensor channels alone.

5. Audit & Industrial EDA

The audit layer generates:

- row/column statistics,
- time ranges,
- target summaries,
- top process correlations.

Correlation analysis currently uses:

```
Pearson correlation
```

through pandas correlation methods.

Purpose:

- fast statistical screening,
- industrial process understanding,
- baseline feature relevance analysis.

However, because MDF processes are nonlinear and interaction-heavy, the architecture evolved beyond pure correlation into:

- ML feature importance,
 - SHAP explainability,
 - and combined explainability scoring.
-

6. Production-Aware Time-Based Cross Validation

A major architectural improvement was the transition from:

- naive random splitting to:
- production-aware time-based validation.

Why?

Because MDF1 production data is NOT IID (Independent and Identically Distributed).

Industrial process data evolves over time:

- raw material changes,
- recipe changes,
- moisture variation,
- operator changes,
- machine behavior drift.

Therefore the pipeline now:

- trains on older production,
- validates on newer production,
- evaluates future generalization behavior.

The service implements:

- chronological train/validation/test splitting,
- expanding-window time-series CV.

This better simulates real industrial deployment conditions and reduces leakage risk.

7. Model Training

For each primary MDF1 lab target:

```
labDensityAverage  
labBendingAvg  
labEModulAvg  
labTensileAvg  
labSurfaceSoundnessAvg
```

the pipeline trains multiple algorithms:

```
Random Forest  
XGBoost  
LightGBM
```

The architecture supports:

- modular algorithm configuration,
- centralized settings,
- reusable training orchestration,
- artifact versioning.

8. Feature Selection

Feature selection is performed ONLY using training data to prevent leakage.

The service:

- selects top correlated features,
- merges domain-forced industrial features,
- removes unstable/invalid signals.

The architecture intentionally combines:

- statistical relevance,
 - domain knowledge,
 - engineered process features.
-

9. XGBoost Hyperparameter Tuning

XGBoost tuning includes:

- learning_rate,
- max_depth,
- gamma,
- subsampling,
- regularization,
- early stopping.

Regularization

The pipeline tunes:

```
reg_alpha  
reg_lambda
```

Purpose:

- reduce overfitting,
- stabilize learning,
- reduce noise memorization,
- improve future production generalization.

Early Stopping

The service uses validation-aware early stopping to stop training when:

- validation performance stops improving.

This prevents:

- over-training,
- noise memorization,
- unstable industrial models.

10. Ensemble Evaluation

The pipeline additionally evaluates:

- XGBoost + Random Forest weighted ensembles.

The ensemble is accepted only if:

- validation performance meaningfully improves over single models.

This introduces:

- industrial robustness,
 - diversified prediction behavior,
 - controlled ensemble governance.
-

11. Explainability Architecture

The explainability layer combines three levels:

A. Correlation

Statistical relationship between process variables and targets.

B. ML Feature Importance

Using:

- Random Forest importance,
- XGBoost importance,
- LightGBM importance.

These measure how strongly trained models rely on features.

C. SHAP Explainability

SHAP computes:

- per-row feature contribution,
- average absolute feature influence,
- global feature importance ranking.

SHAP ranking is based on:

```
mean_abs_shap = np.abs(shap_values).mean(axis=0)
```

Meaning:

- average prediction influence magnitude.

The architecture intentionally combines:

- correlation,
- ML importance,
- SHAP influence

into a final explainability layer.

This provides:

- statistical insight,
 - model insight,
 - prediction-behavior insight.
-

12. Artifact Publishing

Each trained model produces a production-ready artifact bundle:

```
model.joblib  
preprocessor.joblib  
features.json  
metrics.json  
metadata.json  
feature_importance.csv  
feature_importance.xlsx
```

Artifacts are stored under:

```
artifacts/<model_name>/
```

These are later consumed by PredictionService.

13. Best Model Registry

The pipeline generates:

```
best_models_registry.json
```

This registry defines:

- which model serves each target,
- whether serving uses:
 - single model,
 - weighted ensemble,
- artifact locations,
- evaluation metrics.

This acts as the bridge between:

- TrainingService and

- PredictionService.
-

14. Logging & Pipeline Governance

The architecture now includes:

- centralized logging,
- decorators,
- timing monitoring,
- structured pipeline execution tracking.

Pipeline decorators automatically log:

- STARTED,
- FINISHED,
- FAILED,
- execution duration.

Logs are written into:

```
logs/training.log
```

This significantly improves:

- traceability,
 - debugging,
 - industrial maintainability.
-

15. Architectural Refactoring Improvements

The service was progressively refactored to:

- reduce redundancy,
- centralize settings,
- separate responsibilities,
- standardize utilities,
- modularize reporting,
- improve maintainability.

Examples:

- settings centralized in `settings.py`
- reusable filesystem utilities
- reusable logging utilities
- decorator-based orchestration
- reusable report writers

- centralized algorithm configuration

This moved the project from:

- notebook-style ML experimentation

toward:

- industrial-grade ML platform architecture.
-

Current Architectural Position

TrainingService now functions as:

industrial offline ML orchestration platform

with:

- MDF1-specific process intelligence,
- production-aware validation,
- explainability layers,
- artifact governance,
- modular industrial architecture,
- and PredictionService integration readiness.

It is significantly more advanced than a basic ML training script and already resembles an enterprise industrial AI pipeline architecture.